

ВРЕМЯ ЧТЕНИЯ: 9 МИНУТ

Оценка чат-ботов LLM: основные метрики и методы тестирования

В этой статье я научу вас, как оценивать LLM-чат-ботов, чтобы понять, достаточно ли они убедительны, чтобы сойти за реальных людей. Но что еще важнее, вы сможете использовать результаты этой оценки, чтобы понять, как улучшить своего LLM-чат-бота, например, изменив промпт или саму LLM-модель.



В этой статье вы узнаете:

- Разницу между оценкой LLM-чат-ботов и стандартной оценкой LLM
- Различные подходы к оценке LLM-диалогов
- Различные типы метрик для оценки LLM-чат-ботов
- Как реализовать оценку LLM-диалогов в коде с использованием DeepEval

Давайте приступим.

Что такое оценка LLM-чат-бота и чем она отличается от оценки LLM?

Оценка LLM-чат-ботов — это процесс анализа производительности разговорных агентов, работающих на основе больших языковых моделей, с целью оценки качества их ответов в ходе диалога. Она отличается от стандартной [оценки LLM \(или LLM-систем\)](#) тем, что в то время как стандартная оценка LLM анализирует приложения на основе отдельных взаимодействий «ввод-вывод», оценка LLM-чат-ботов включает в себя анализ взаимодействий с учетом истории предыдущих сообщений как дополнительного контекста.

Разница между оценкой чат-бота LLM и оценкой системы LLM

Несмотря на то, что критерии оценки LLM-чат-бота могут быть чрезвычайно похожими для неразговорчивых агентов LLM, метрики, используемые для проведения этой оценки, требуют совершенно другой реализации, чтобы учитывать историю диалога.

Эта «история диалога» имеет более технический термин — ходы. Когда вы слышите о многооборотном разговоре, это, по сути, просто причудливый способ описания обмена сообщениями между пользователем и LLM-чат-ботом.

Таким образом, возникает вопрос: имея список ходов в диалоге, что и как мы должны оценивать? Стоит ли анализировать диалог в целом, отдельные ходы в диалоге, или что-то еще?

Различные режимы оценки диалогов LLM

- **Оценка всего диалога:** включает анализ всей беседы на основе всех поворотов, происходящих в рамках одного диалога.
- **Оценка последнего и наиболее значимого ответа:** предполагает оценку только последнего ответа, сгенерированного чат-ботом LLM в ходе диалога.

Причина, по которой мы не оцениваем отдельные ходы, заключается в том, что часто возникает много ненужной болтовни, а это является пустой тратой токенов для оценки. Принцип остается тем же: какой бы тип оценки диалога вы ни решили принять, он должен проводиться с учетом контекста предыдущих ходов.

Оценка всего диалога

Оценка всего диалога в целом полезна, потому что некоторые критерии оценки требуют весь диалог в качестве контекста. Представьте, что вы создаете чат-бота, чтобы помочь пользователям открыть банковский счет. Такой агент на основе LLM должен будет спросить имя пользователя, адрес, номер социального страхования (SSN) и другую информацию, но распространенная проблема, с которой вы можете столкнуться, заключается в том, что агент может забывать уже предоставленную пользователем информацию, что приводит к повторным вопросам и вызывает у пользователей раздражение.

Чтобы определить метрику для оценки способности LLM и промптов сохранять ранее полученные данные, вам придется анализировать весь диалог целиком. Первая метрика, которую я хотел бы представить, — это **метрика сохранения знаний** (knowledge retention metric). Она оценивает, насколько хорошо чат-бот на основе LLM способен сохранять предоставленную информацию на протяжении всего диалога. Вы можете получить к ней доступ в DeepEval.

Еще одна ситуация, в которой оценка всего диалога полезна, — это когда вы хотите использовать все ходы в беседе для построения итогового оценочного показателя. (Если вас интересует, что я имею в виду под «оценочным показателем», [рекомендую прочитать статью, где подробно объясняется, что такое метрика оценки LLM.](#))

Например, вы хотите измерить, насколько релевантны ответы чат-бота. Итоговая метрика релевантности диалога может быть определена как количество релевантных ответов, деленное на общее количество ходов в беседе. Но возникает вопрос: как определить, является ли ответ релевантным?

Разве вы не можете просто попросить судью LLM назвать вам количество нерелевантных ответов, предоставив ему всю историю ходов? Проблема с этим подходом состоит в том, что [судья LLM может галлюцинировать](#), когда разговор становится длинным. Но еще более важный вопрос, который вы должны задать себе, — какие предыдущие ходы следует учитывать при измерении релевантности ответа. Позвольте мне объяснить.

Представьте диалог из 100 ходов (да, это звучит как задача из учебника по математике). Вы оцениваете 50-й ход. Ответ может быть нерелевантным, если учитывать только два предыдущих хода (48-й и 49-й). Однако он оказывается крайне релевантным, если принять во внимание предыдущие 10 ходов (с 39-го по 49-й).

Метод оценки скользящего окна для учета предыдущих ходов

Но что, если релевантность ответа становится очевидной только при учете 30 предыдущих ходов? 100? Или даже 1000? Таким образом, главный вопрос: сколько предыдущих ходов нужно учитывать при оценке конкретного ответа LLM-чат-бота? Вот почему при оценке целых разговоров в некоторых случаях важно определить, какую часть истории предыдущего разговора следует учитывать.

Те, кто имеет более традиционный опыт разработки программного обеспечения, поймут, что эту проблему можно решить, используя метод скользящего окна, где для каждого ответа мы можем принять во внимание предыдущий минимум (0, текущий номер хода — размер окна) ходов, чтобы определить релевантность ответа. Возвращаясь к примеру метрики релевантности, для расчета итогового показателя релевантности мы выполняем следующие шаги:

- Проходим через каждый отдельный ход в диалоге
- Используя подход скользящего окна, захватываем последний минимум (0, номер текущего хода — размер окна) ходов и учитываем их при определении релевантности текущего ответа
- Суммируем количество релевантных ответов и делим это число на общее количество ходов

Давайте перейдем к более простому способу оценки диалога.

Оценка последнего лучшего ответа

Некоторые пользователи вместо оценки всего разговора могут предпочесть оценить последний лучший ответ в разговоре. Это не значит, что предыдущие ходы в разговоре не учитываются как контекст. Просто, вместо того чтобы анализировать ответы из всего диалога, мы рассматриваем только последний ответ и вычисляем его метрику на основе

Подход к оценке последнего лучшего ответа

Хорошая новость для людей, желающих оценить последний лучший ответ в разговоре, заключается в том, что вы можете повторно использовать ВСЕ неразговорные метрики оценки LLM, используемые для оценки отдельных ответов системы LLM. Но на самом деле вам придется немного подправить их, чтобы включить некоторое количество предыдущих ходов в качестве дополнительного контекста. Однако это значительно проще, чем проводить оценку полного диалога.

Оценка разговора LLM с помощью DeepEval

DeepEval - это [фреймворк для оценки LLM с открытым исходным кодом](#). В DeepEval вы можете оценивать диалоги LLM, создавая [тестовые кейсы](#) для диалогов. Такой тестовый кейс состоит из списка ходов, которые представляют ваш диалог с моделью LLM.

```
pip install deepeval
```

```
from deepeval.test_case import ConversationalTestCase

convo_test_case = ConversationalTestCase(
    chatbot_role="You are a happy jolly CS agent",
    turns=[
```

```
LLMTestCase(input="Hi", actual_output="Hey how can I help?"),
LLMTestCase(input="I just want someone to talk to", actual_output="No problem")
]
```

Ходы диалога представляют собой экземпляры класса **LLMTestCases**, где **input** — это запрос пользователя, а **actual_output** — ответ чат-бота на основе LLM. Также существует параметр **chatbot_role**, который нужен только в том случае, если вы собираетесь использовать метрику соблюдения роли (role adherence metric) на более поздних этапах. Подробнее о тестовых кейсах для диалогов и LLM можно узнать [здесь](#).

После того как вы оформили диалог в виде тестового кейса, остается только импортировать нужную метрику и применить ее для оценки тестового кейса. Вот пример:

```
from deepeval.metrics import KnowledgeRetentionMetric

...
metric = KnowledgeRetentionMetric(verbose_mode=True)
metric.measure(convo_test_case)
print(metric.score, metric.reason)
```

Это на самом деле так просто.

Различные типы метрик оценки чат-ботов LLM

В этом разделе я расскажу о четырех наиболее полезных метриках для оценки диалогов, которые вам стоит рассмотреть при анализе полных диалогов, а также покажу, как использовать их с помощью менее чем пяти строк кода через [DeepEval](#).

Эти метрики специально разработаны для оценки всего диалога. Поэтому, если вы хотите оценивать диалоги, используя подход «последний лучший ответ» (last best response), [обратите внимание на подробную статью, которую я написал о метриках оценки LLM](#). В ней рассматриваются все метрики, которые могут понадобиться для оценки последних ответов, включая RAG, агентов LLM, а также кейсы, связанные с дообучением. В статье также объясняется, почему во всех метриках, связанных с оценкой LLM, [мы используем подход «LLM-as-a-judge»](#).

(P.S. DeepEval также предоставляет более 14 предопределенных метрик оценки LLM и

Соответствие роли

[Метрика соответствия роли](#) оценивает, способен ли ваш чат-бот LLM действовать так, как ему поручено, на протяжении всего разговора. Эта метрика особенно полезна для сценариев, связанных с ролевыми играми. Она рассчитывается путем последовательного анализа каждого хода в диалоге с использованием LLM для определения, какой из них не соответствует заданной роли чат-бота, при этом в качестве контекста используются предыдущие шаги. Итоговый показатель метрики соответствия роли вычисляется как отношение количества ходов, соответствующих заданной роли, к общему числу ходов в тестовом кейсе диалога.

```
from deepeval.test_case import ConversationalTestCase, LLMTestCase
from deepeval.metrics import RoleAdherenceMetric

convo_test_case = ConversationalTestCase(
    chatbot_role="description of chatbot role goes here",
    turns=[
        LLMTestCase(
            input="user input goes here",
            actual_output="LLM chatbot repsonse goes here"
        )
    ]
)
metric = RoleAdherenceMetric(verbose_mode=True)
metric.measure(convo_test_case)
print(metric.score, metric.reason)
```

Релевантность разговора

[Метрика релевантности разговора](#), представленная ранее, оценивает, способен ли ваш чат-бот LLM генерировать релевантные ответы на протяжении всего разговора. Она рассчитывается путем последовательного анализа каждого хода в диалоге и использует метод скользящего окна, чтобы учитывать последний минимум (0, текущий номер хода — размер окна) ходов для определения его релевантности. Итоговая оценка релевантности разговора — это количество релевантных ответов, деленное на общее количество ходов в тестовом кейсе диалога.

```
from deepeval.test_case import ConversationalTestCase, LLMTestCase
from deepeval.metrics import ConversationRelevancyMetric

convo_test_case = ConversationalTestCase(
    turns=[
        LLMTestCase(
            input="user input goes here",
            actual_output="LLM chatbot repsonse goes here"
        )
    ]
)
metric = ConversationRelevancyMetric(window_size=
5
, verbose_mode=True)
metric.measure(convo_test_case)
print(metric.score, metric.reason)
```

Удержание знаний

[Метрика удержания знаний](#) оценивает, насколько эффективно чат-бот на основе LLM сохраняет информацию, предоставленную ему в ходе беседы. Рассчитывается она следующим образом: сначала формируется список знаний, переданных боту до определенного хода в диалоге, а затем проверяется, запрашивает ли LLM информацию, которая уже содержится в ответах на предыдущих ходах. Итоговый показатель удержания знаний вычисляется как отношение числа ходов, на которых не было утраты знаний, к общему количеству ходов в тестовом кейсе.

```
from deepeval.test_case import ConversationalTestCase, LLMTestCase
from deepeval.metrics import KnowledgeRetentionMetric

convo_test_case = ConversationalTestCase(
    turns=[
        LLMTestCase(
            input="user input goes here",
            actual_output="LLM chatbot repsonse goes here"
        )
    ]
)
metric = KnowledgeRetentionMetric(verbose_mode=True)
```


Полнота разговора

[Метрика полноты разговора](#) оценивает, насколько эффективно чат-бот на основе LLM выполняет запросы пользователя в ходе беседы. Она полезна, поскольку полнота разговора может служить косвенным показателем для измерения удовлетворенности пользователя и эффективности чат-бота. Метрика рассчитывается следующим образом: сначала с помощью LLM извлекается список основных намерений пользователя, выявленных в ходе разговора, а затем с помощью того же LLM определяется, были ли эти намерения выполнены и/или удовлетворены в ходе беседы.

```
from deepeval.test_case import ConversationalTestCase, LLMTestCase
from deepeval.metrics import ConversationCompletenessMetric

convo_test_case = ConversationalTestCase(
    turns=[
        LLMTestCase(
            input="user input goes here",
            actual_output="LLM chatbot repsonse goes here"
        )
    ]
)
metric = ConversationCompletenessMetric(verbose_mode=True)
metric.measure(convo_test_case)
print(metric.score, metric.reason)
```

Использование DeepEval с Confident AI для регрессионного тестирования чат-ботов LLM

Хотя DeepEval представляет собой фреймворк для оценки LLM, Confident AI — это платформа для оценки LLM, основанная на DeepEval. Использование Confident AI является необязательным, но она предоставляет возможность запускать метрики DeepEval в облаке и генерирует отчеты о тестировании, которые помогут вам выявить регрессии в LLM между несколькими оценками.

При необходимости войдите в систему через DeepEval через интерфейс командной строки:

deepeval login

И запустите оценку для регрессионного тестирования вашего чат-бота LLM, используя ту же архитектуру, которую мы обсуждали выше:

```
from deepeval import evaluate
from deepeval.test_case import ConversationalTestCase, LLMTestCase
from deepeval.metrics import ConversationRelevancyMetric

convo_test_case = ConversationalTestCase(
    turns=[
        LLMTestCase(
            input="user input goes here",
            actual_output="LLM chatbot repsonse goes here"
        )
    ]
)
metric = ConversationRelevancyMetric()

evaluate(test_cases=[convo_test_case], metrics=[metric])
```

Вы локально получите доступ к результатам после выполнения оценки, и если вы используете Confident AI, будет создан отчет о тестировании, который вы можете использовать для сравнения результатов тестирования.

Регрессионное тестирование чат-ботов LLM на Confident AI

Заключение

Теперь у вас есть все, что нужно знать, чтобы начать оценивать чат-ботов LLM. В этой статье мы рассмотрели способы оценки разговоров с LLM, различные техники, которые можно применить, какие метрики следует отслеживать и как реализовать это с помощью DeepEval.

Автор оригинала: [Jeffrey Ip](#)

Поможем
выбрать
лучшую

Хотите бесплатно
проконсультироваться по
вашему проекту?



Читать далее

Будущее, в котором ИИ — герой, а не злодей

Эра искусственного интеллекта уже наступила — и люди, мягко говоря, в панике.

К счастью, я здесь, чтобы рассказать хорошие новости: AI не уничтожит мир, а наоборот, вполне может его спасти.

07.08.2025 СТАТЬИ

MCP — новый кирпичик в фундаменте AI-разработки

Model Context Protocol, расширяющий возможности AI в IDE, стремительно набирает популярность. Почему это происходит — и почему разработчикам стоит обратить на него внимание?

04.08.2025 СТАТЬИ

AI-агенты в деле: 15 рабочих примеров для роста вашего бизнеса

AI-агенты радикально меняют парадигму работы технических команд, переходя от традиционных, основанных на правилах workflow к более динамичным, интеллектуальным системам, способным адаптироваться и принимать решения в реальном времени.

23.07.2025 СТАТЬИ

info@llmarena.ru
+7 (915) 318 09-64

Ищите нас в:

Telegram

Hugging
Face

LLMArena в
TG